

Méta-algorithmes

Bagging, Forêts aléatoires, Boosting, Stacking

STA211 — Apprentissage supervisé — Fiche récapitulative

6 juin 2026

Table des matières

1	Présentation	2
2	Bagging	3
2.1	Principe	3
2.2	Propriétés	3
2.3	Erreur OOB	3
2.4	Vote majoritaire	3
3	Forêts aléatoires	4
3.1	Principe	4
3.2	Choix des hyperparamètres	4
3.3	Importance des variables	4
4	Boosting	5
4.1	Principe général	5
4.2	AdaBoost	5
4.3	Gradient Boosting	5
4.4	XGBoost, LightGBM, CatBoost	6
5	Stacking	7
5.1	Principe	7
5.2	Procédure	7
5.3	Choix du méta-modèle	7
6	Comparaison des méthodes	8
7	Code Python	9
8	Code R	10
9	QCM	11

1 Présentation

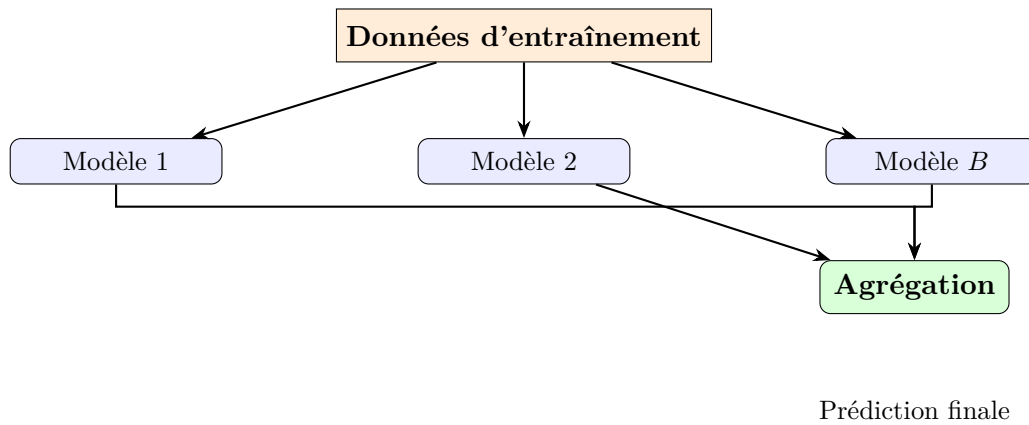
Les **méta-algorithmes** (ou méthodes d'ensemble) combinent plusieurs modèles faibles (*weak learners*) pour obtenir un prédicteur plus stable et plus performant. L'idée est qu'un groupe de modèles décide mieux qu'un seul.

Principe général :

- On entraîne B modèles de base $\hat{f}_1, \dots, \hat{f}_B$ sur des versions perturbées des données.
- La prédiction finale agrège les B prédictions.

Agrégation :

- **Régression** : $\hat{f}_{\text{ens}}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(\mathbf{x})$ (moyenne)
- **Classification** : $\hat{f}_{\text{ens}}(\mathbf{x}) = \arg \max_k \sum_{b=1}^B \mathbf{1}_{\{\hat{f}_b(\mathbf{x})=k\}}$ (vote majoritaire)



	Méthode	Perturbation	Modèles de base
Principales méthodes :	Bagging	Bootstrap (échantillonnage)	Arbres profonds
	Forêts aléatoires	Bootstrap + sous-échantillonnage des variables	Arbres profonds
	Boosting	Pondération des individus	Arbres peu profonds
	Stacking	Partitionnement des données	Modèles variés

2 Bagging

2.1 Principe

Le **Bagging** (*Bootstrap AGGREGatING*) — Breiman, 1996 — consiste à :

1. Générer B échantillons bootstrap (tirage avec remise de n individus parmi n) à partir des données originales.
2. Entraîner un modèle $\hat{f}_b$ sur chaque échantillon.
3. Agréger les B prédictions (moyenne ou vote).

2.2 Propriétés

- **Réduction de la variance** : pour des modèles identiquement distribués avec variance σ^2 et corrélation deux à deux ρ ,

$$\mathbb{V}\left[\frac{1}{B} \sum_{b=1}^B \hat{f}_b\right] = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

Quand $B \rightarrow \infty$, la variance tend vers $\rho\sigma^2$.

- Le bagging est surtout efficace pour des modèles **instables** (forte variance, faible biais) comme les arbres CART non élagués.
- Chaque arbre bootstrap utilise environ 63,2% des données (probabilité de tirage de chaque individu). Les 36,8% restants sont les **Out-Of-Bag (OOB)**.

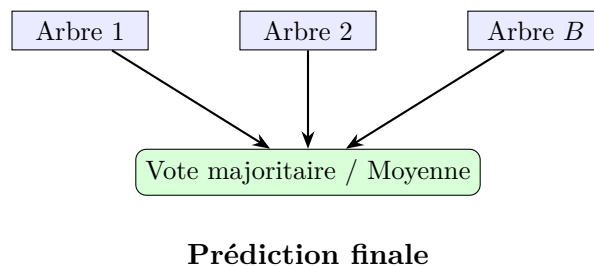
2.3 Erreur OOB

On estime l'erreur de généralisation sans validation croisée :

$$\text{Err}_{\text{OOB}} = \frac{1}{n} \sum_{i=1}^n \mathcal{L}\left(y_i, \frac{1}{|\mathcal{B}_i|} \sum_{b \in \mathcal{B}_i} \hat{f}_b(\mathbf{x}_i)\right)$$

où $\mathcal{B}_i$ est l'ensemble des arbres pour lesquels l'individu i est OOB.

2.4 Vote majoritaire



3 Forêts aléatoires

3.1 Principe

Les **forêts aléatoires** (*Random Forests*) — Breiman, 2001 — améliorent le bagging en **décorrélant** les arbres. En plus du bootstrap sur les individus, on tire aléatoirement un sous-ensemble de m variables candidates à chaque division.

Algorithme :

1. Pour $b = 1, \dots, B$:
 - (a) Tirer un échantillon bootstrap $\mathcal{D}_b$ de taille n .
 - (b) Construire un arbre CART profond (non élagué) sur $\mathcal{D}_b$:
 - À chaque nœud, sélectionner m variables parmi p au hasard.
 - Choisir la meilleure division parmi ces m variables.
2. Agréger les B arbres (moyenne pour régression, vote pour classification).

3.2 Choix des hyperparamètres

Paramètre	Valeur typique	Effet
B (nombre d'arbres)	100 – 1000	Plus grand $\Rightarrow$ plus stable
m (variables par nœud)	$\sqrt{p}$ (classif.) / $p/3$ (régr.)	Petit $\Rightarrow$ décorrélant
Profondeur max	Illimitée (default)	Arbres profonds = faible biais
Taille minimale des feuilles	1 – 5	Évite le surajustement

3.3 Importance des variables

On mesure l'importance d'une variable X_j en cumulant la baisse de critère (Gini ou RSS) dans tous les nœuds où X_j est utilisée :

$$\text{Imp}(X_j) = \sum_{\text{nœuds } t \text{ divisés par } X_j} \Delta_t$$

où Δ_t est la réduction d'hétérogénéité apportée par la division au nœud t .

4 Boosting

4.1 Principe général

Le **boosting** est une approche **séquentielle** : on entraîne les modèles les uns après les autres, chaque nouveau modèle corrigeant les erreurs du précédent.

Contrairement au bagging (parallélisable), le boosting est **séquentiel** et donc plus lent.

4.2 AdaBoost

Freund & Schapire, 1997. AdaBoost (*Adaptive Boosting*) pondère les individus par leur difficulté :

Algorithme (classification binaire $y_i \in \{-1, 1\}$) :

1. Initialiser les poids : $w_i^{(1)} = 1/n$ pour $i = 1, \dots, n$.
2. Pour $b = 1, \dots, B$:
 - (a) Entraîner un classifieur faible $\hat{f}_b$ sur les données pondérées par $\mathbf{w}^{(b)}$.
 - (b) Calculer l'erreur pondérée : $\varepsilon_b = \frac{\sum_{i=1}^n w_i^{(b)} \mathbf{1}_{\{\hat{f}_b(\mathbf{x}_i) \neq y_i\}}}{\sum_{i=1}^n w_i^{(b)}}$
 - (c) Calculer le poids du classifieur : $\alpha_b = \frac{1}{2} \log\left(\frac{1 - \varepsilon_b}{\varepsilon_b}\right)$
 - (d) Mettre à jour les poids : $w_i^{(b+1)} = w_i^{(b)} \exp[-\alpha_b y_i \hat{f}_b(\mathbf{x}_i)]$, puis normaliser.
3. Prédiction finale : $\hat{f}(\mathbf{x}) = \text{signe}\left(\sum_{b=1}^B \alpha_b \hat{f}_b(\mathbf{x})\right)$

Les poids augmentent pour les individus mal classés, forçant le prochain classifieur à se concentrer sur les cas difficiles.

4.3 Gradient Boosting

Friedman, 2001. Le Gradient Boosting généralise AdaBoost à n'importe quelle fonction de perte différentiable.

Algorithme :

1. Initialiser : $\hat{f}^{(0)}(\mathbf{x}) = \arg \min_{\gamma} \sum_{i=1}^n \mathcal{L}(y_i, \gamma)$
2. Pour $b = 1, \dots, B$:
 - (a) Calculer les résidus (gradient négatif) :

$$r_i^{(b)} = - \left. \frac{\partial \mathcal{L}(y_i, f(\mathbf{x}_i))}{\partial f(\mathbf{x}_i)} \right|_{f=\hat{f}^{(b-1)}}$$
 - (b) Ajuster un arbre faible $\hat{f}_b$ aux résidus $\{(\mathbf{x}_i, r_i^{(b)})\}_{i=1}^n$.
 - (c) Mettre à jour : $\hat{f}^{(b)}(\mathbf{x}) = \hat{f}^{(b-1)}(\mathbf{x}) + \eta \cdot \hat{f}_b(\mathbf{x})$, où η est le taux d'apprentissage.

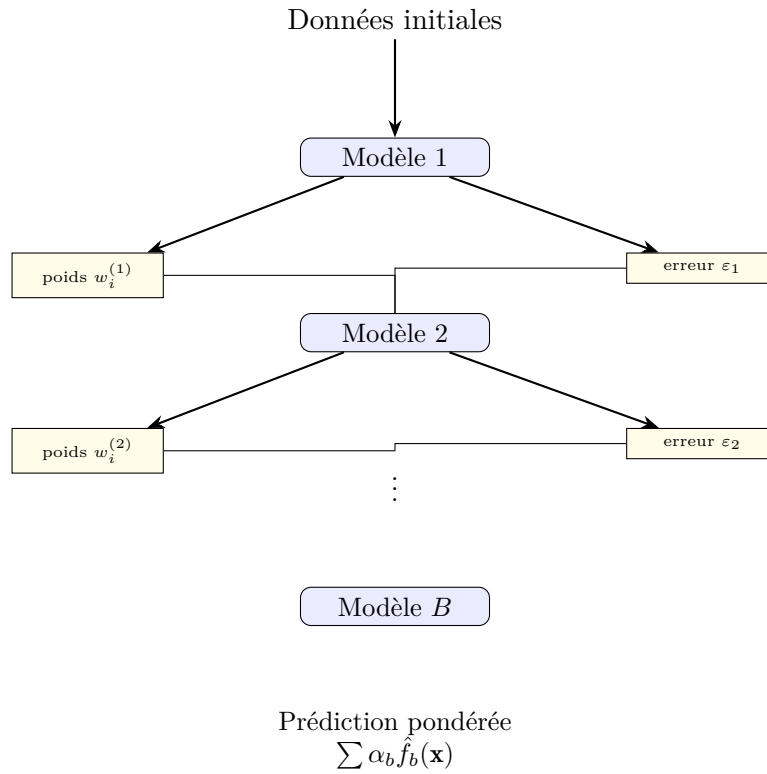
Hyperparamètres importants :

- B : nombre d'itérations (arbres).
- η (taux d'apprentissage) : typiquement 0,01 – 0,3. Plus η est petit, plus il faut d'arbres.
- Profondeur des arbres : souvent 1–3 (arbres peu profonds = faible variance).

4.4 XGBoost, LightGBM, CatBoost

Ce sont des implémentations optimisées du Gradient Boosting :

Méthode	Caractéristique	Avantage
XGBoost	Régularisation L1/L2, pruning	Généralisation, rapidité
LightGBM	Feuilles (<i>leaf-wise</i>)	Très rapide, grandes données
CatBoost	Variables catégorielles	Pas de prétraitement



5 Stacking

5.1 Principe

Le **Stacking** (*Stacked Generalization*) — Wolpert, 1992 — combine des modèles **de nature différente** en utilisant un **méta-modèle** qui apprend à agréger leurs prédictions.

Niveaux :

- **Niveau 0** (base) : K modèles différents $\hat{f}_1, \dots, \hat{f}_K$ (ex. : arbre, régression logistique, SVM, kNN).
- **Niveau 1** (méta) : un modèle $\hat{g}$ qui prend en entrée les prédictions du niveau 0 et apprend à les combiner.

5.2 Procédure

Pour éviter le surajustement, on utilise la **validation croisée** pour générer les entrées du méta-modèle :

1. Diviser les données en V plis.
2. Pour chaque pli v :
 - (a) Entraîner les K modèles de base sur les $V - 1$ autres plis.
 - (b) Prédire sur le pli v avec ces K modèles.
3. On obtient une matrice $\mathbf{Z}$ de taille $n \times K$: prédictions (hors-échantillon) de chaque modèle de base.
4. Entraîner le méta-modèle $\hat{g}$ sur $(\mathbf{Z}, \mathbf{y})$.

5.3 Choix du méta-modèle

Le méta-modèle est souvent simple pour éviter le surajustement :

- Régression logistique (classification)
- Régression linéaire (régression)
- Un petit arbre

6 Comparaison des méthodes

Critère	Bagging / RF	Boosting	Stacking
Entraînement	Parallèle	Séquentiel	Parallèle (base) + méta
Risque de surajustement	Faible (RF)	Élevé si B trop grand	Modéré (K pas trop grand)
Interprétabilité	Faible	Faible	Très faible
Performance	Bonne	Très bonne	Potentiellement meilleure
Popularité	Très utilisé	Très utilisé	Peu utilisé

Biais Bagging / RF **Biais élevé, variance faible**

Variance Boosting **Biais faible, variance élevée**

7 Code Python

```

1 from sklearn.ensemble import (
2     BaggingRegressor, RandomForestRegressor,
3     GradientBoostingRegressor, StackingRegressor
4 )
5 from sklearn.tree import DecisionTreeRegressor
6 from sklearn.linear_model import LinearRegression
7 from sklearn.svm import SVR
8 from sklearn.model_selection import train_test_split, cross_val_predict
9 from sklearn.metrics import mean_squared_error
10 import numpy as np, pandas as pd
11
12 # --- Données ---
13 np.random.seed(42)
14 X = np.random.randn(200, 5)
15 y = X[:, 0] + 2 * X[:, 1] + np.sin(X[:, 2]) + 0.5 * np.random.randn(200)
16 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
17
18 # --- Bagging ---
19 bag = BaggingRegressor(
20     estimator=DecisionTreeRegressor(max_depth=None),
21     n_estimators=100, random_state=42
22 )
23 bag.fit(X_train, y_train)
24 print(f"Bagging RMSE: {mean_squared_error(y_test, bag.predict(X_test), squared=False):.3f}")
25
26 # --- Forêts aleatoires ---
27 rf = RandomForestRegressor(
28     n_estimators=200, max_depth=None,
29     max_features='sqrt', random_state=42
30 )
31 rf.fit(X_train, y_train)
32 print(f"RF RMSE: {mean_squared_error(y_test, rf.predict(X_test), squared=False):.3f}")
33 print(f"Importance: {rf.feature_importances_.round(3)}")
34
35 # --- Gradient Boosting ---
36 gb = GradientBoostingRegressor(
37     n_estimators=200, learning_rate=0.1,
38     max_depth=3, random_state=42
39 )
40 gb.fit(X_train, y_train)
41 print(f"GB RMSE: {mean_squared_error(y_test, gb.predict(X_test), squared=False):.3f}")
42
43 # --- Stacking ---
44 base_models = [
45     ('rf', RandomForestRegressor(n_estimators=100, random_state=42)),
46     ('svr', SVR(kernel='rbf')),
47     ('lr', LinearRegression())
48 ]
49 meta = LinearRegression()
50 stack = StackingRegressor(
51     estimators=base_models, final_estimator=meta,
52     cv=5
53 )
54 stack.fit(X_train, y_train)
55 print(f"Stacking RMSE: {mean_squared_error(y_test, stack.predict(X_test), squared=False):.3f}")

```

Listing 1 – Bagging

8 Code R

```

1 library(rpart)
2 library(randomForest)
3 library(gbm)
4 library(caret)
5
6 # --- Donnees ---
7 set.seed(42)
8 n <- 200
9 X <- matrix(rnorm(n * 5), n, 5)
10 colnames(X) <- paste0("X", 1:5)
11 y <- X[,1] + 2*X[,2] + sin(X[,3]) + 0.5*rnorm(n)
12 dat <- data.frame(y, X)
13
14 # --- Bagging (rpart avec bootstrap manuel ou caret) ---
15 bag <- train(y ~ ., data = dat, method = "treebag",
16             trControl = trainControl(method = "boot", number = 50))
17 print(bag)
18
19 # --- Foret aleatoire ---
20 rf <- randomForest(y ~ ., data = dat, ntree = 200, importance = TRUE)
21 print(rf)
22 importance(rf)
23
24 # --- Gradient Boosting ---
25 gb <- gbm(y ~ ., data = dat, distribution = "gaussian",
26           n.trees = 200, interaction.depth = 3,
27           shrinkage = 0.1, cv.folds = 5)
28 best <- gb.perf(gb, method = "cv")
29 summary(gb, n.trees = best)

```

Listing 2 – Bagging

9 QCM

1. **Quel est le principal avantage du bagging par rapport à un seul arbre CART ?**
 - (a) Réduction du biais
 - (b) Réduction de la variance
 - (c) Meilleure interprétabilité
 - (d) Entraînement plus rapide
2. **Dans une forêt aléatoire, le paramètre m (nombre de variables candidates par nœud) permet de :**
 - (a) Réduire le biais
 - (b) Décorréliser les arbres
 - (c) Accélérer l'entraînement
 - (d) Éviter le surajustement par élagage
3. **En classification, la valeur typique de m dans une forêt aléatoire est :**
 - (a) $m = p$
 - (b) $m = \sqrt{p}$
 - (c) $m = p/3$
 - (d) $m = \log_2 p$
4. **Contrairement au bagging, le boosting est un algorithme :**
 - (a) Parallélisable
 - (b) Séquentiel
 - (c) Non supervisé
 - (d) Basé sur des réseaux de neurones
5. **Dans AdaBoost, le poids α_b d'un classifieur dépend :**
 - (a) Du nombre d'arbres B
 - (b) De son erreur pondérée ε_b
 - (c) De la profondeur de l'arbre
 - (d) Du taux d'apprentissage η
6. **Le Gradient Boosting diffère d'AdaBoost car il :**
 - (a) Utilise des arbres profonds
 - (b) Généralise à toute fonction de perte différentiable via le gradient
 - (c) Est parallélisable
 - (d) N'utilise pas de pondération des individus
7. **Dans le stacking, le méta-modèle est entraîné sur :**
 - (a) Les données originales
 - (b) Les prédictions des modèles de base (hors-échantillon)
 - (c) Les résidus du meilleur modèle
 - (d) Les variables les plus importantes
8. **L'erreur OOB (Out-Of-Bag) est calculée sur :**
 - (a) Les données de test
 - (b) Les individus non tirés dans chaque échantillon bootstrap
 - (c) L'échantillon d'apprentissage complet
 - (d) Les feuilles de l'arbre
9. **Parmi ces implémentations du Gradient Boosting, laquelle régularise avec les normes L1/L2 ?**
 - (a) CatBoost

- (b) LightGBM
 - (c) XGBoost
 - (d) AdaBoost
10. **Pourquoi les forêts aléatoires fonctionnent-elles mieux que le simple bagging d'arbres ?**
- (a) Parce qu'elles utilisent plus d'arbres
 - (b) Parce qu'elles décorrèlent les arbres en sous-échantillonnant les variables
 - (c) Parce qu'elles élaguent les arbres
 - (d) Parce qu'elles utilisent un méta-modèle

Réponses au QCM

N°	Réponse	Explication
1	b	Le bagging réduit la variance en moyennant des modèles entraînés sur des bootstrap.
2	b	$m < p$ force les arbres à utiliser des variables différentes, ce qui les décorrèle.
3	b	$m = \sqrt{p}$ est la valeur par défaut recommandée pour la classification.
4	b	Le boosting est séquentiel : chaque modèle corrige les erreurs du précédent.
5	b	$\alpha_b = \frac{1}{2} \log((1 - \varepsilon_b)/\varepsilon_b)$: plus l'erreur est faible, plus le poids est grand.
6	b	Le Gradient Boosting utilise le gradient négatif de n'importe quelle fonction de perte.
7	b	Le méta-modèle apprend à combiner les prédictions de base obtenues par CV.
8	b	L'erreur OOB utilise les 36,8 % d'individus non tirés dans chaque bootstrap.
9	c	XGBoost ajoute une régularisation L1/L2 dans sa fonction objectif.
10	b	Le sous-échantillonnage des variables ($m < p$) décorrèle les arbres, réduisant la variance de l'ensemble.

Fin de la fiche.